&&

AITW

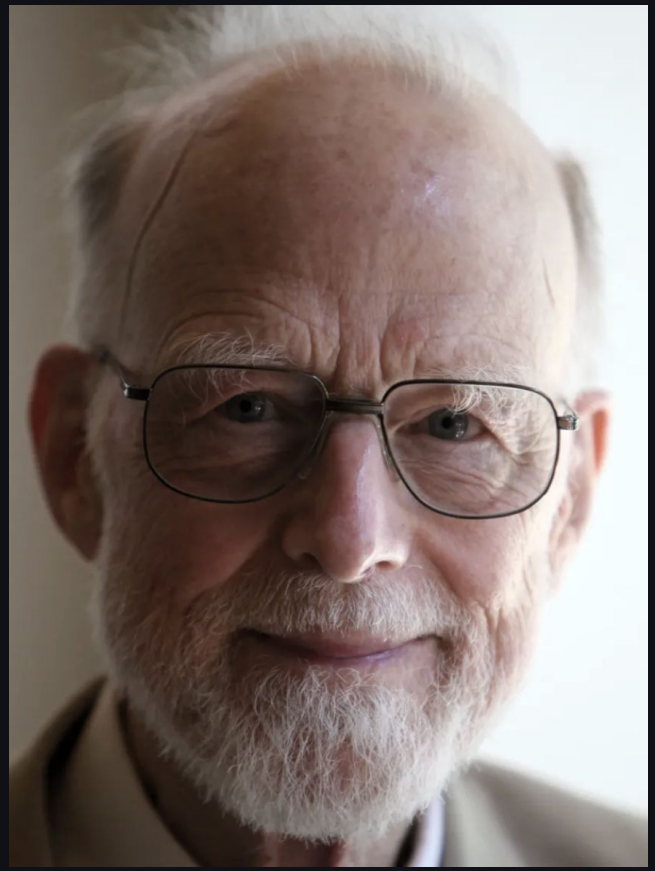
CONFERENCE

31 MARZO 2026 | ROMA

L’evento dove si incontra il futuro tech: developer, tech leader e AI innovator

Le crepe del paradigma perfetto

OOP — promesse, limiti, e cosa viene dopo



Tony Hoare

11 Gennaio 1936, 5 Marzo 2026

Quante volte hai scritto

extends

questa settimana?

Quante volte ti sei chiesto

se fosse la scelta giusta?

ACT 0

Mappa del talk

ACT 01

Le promesse

Da dove veniamo

ACT 02

Le crepe

4 problemi strutturali

ACT 03

I pattern

Sintomi, non cure

ACT 04

Il futuro

Cosa portare a casa

1967

Oslo, Norvegia

Ole-Johan Dahl & Kristen Nygaard creano Simula 67 — il primo linguaggio con oggetti e classi.

Non stavano inventando un paradigma.
Stavano risolvendo un problema concreto.

Simula 67

Il catechismo

Encapsulation

Nascondi lo stato. Esponi solo quello che serve.

Inheritance

Riusa il codice. Non riscrivere quello che hai già.

Polymorphism

Scrivi codice che funziona su tipi diversi.

ACT 1

Perché ci ha convinto

Il mondo è fatto di oggetti. Un cane ha stato — nome, età, razza.
Ha comportamento — abbaia, mangia, corre.

> Questa intuizione è vera — ma solo fino a un certo punto.
> Il mondo reale è complicato. Le gerarchie rigide scricchiolano.

ACT 1

Il diagramma che tutti abbiamo disegnato

Bella. Finché non arriva il business.

E poi è arrivato il business

```
WorkingDog →  Dog  o  ServiceAnimal?

ServiceAnimal →  Animal  o  interfaccia?

HybridPet →  ???
```

> L'OOP ci ha dato gli strumenti per costruire la lavagna.
> Non ci ha dato gli strumenti per capire quando smettere.

1995

Java e il mainstream

Java non ha inventato l'OOP. Ma l'ha portato ovunque.
Nelle università, nei libri, nelle aziende.

Quello che tutti insegnano diventa
quello che tutti danno per scontato.

1994 — Design Patterns, p. 20

"Favor object composition over class inheritance."

Lo abbiamo letto. Lo abbiamo sottolineato.
Lo abbiamo messo sullo scaffale.

E poi siamo andati avanti a scrivere **`extends`**.

2026

AbstractBaseServiceFactoryImpl

CREPA #1

Fragile Base Class

Una conseguenza strutturale dell'ereditarietà

ACT 2

La gerarchia

```
class Collection {
  add(element) { ... }

  addAll(list) {
    // itera e chiama add() per ogni elemento
    list.forEach(e => this.add(e))
  }
}
```

ACT 2

La sottoclasse innocente

```
class InstrumentedCollection extends Collection {
  count = 0

  add(e) { count++; super.add(e) }
  addAll(list) {
     count += list.size();
     super.addAll(list)
  }
```

Sembra corretto. Vero?

```
addAll([a, b, c])

count → 6
```

addAll() della superclasse chiama add() internamente.
Il nostro override conta due volte.

Non abbiamo fatto nulla di sbagliato.
Abbiamo usato l'ereditarietà esattamente come ci hanno insegnato.

> "L'ereditarietà espone i dettagli implementativi.
> Non è encapsulation — è il contrario."

```
java.util.Stack extends Vector
```

Stack espone `get(index)`, `insertElementAt()`, `remove(index)`

Trent'anni di debito tecnico in una singola riga di ereditarietà.

Virtual Methods

Il costo dell'indirezione non è nullo (anzi)

CREPA #2

The Billion Dollar Mistake

null

"I call it my billion-dollar mistake.
It was the invention of the null reference in 1965."

Tony Hoare

QCon London, 2009

L'inventore di Quicksort.

ACT 2

Perché null è un problema di design

Cosa pensi

```
user.getAddress().getCity()
```

Cosa succede

```
NullPointerException
```

Runtime. In produzione. A mezzanotte.

```
User getUser(String id)
```

Mente.

Restituisce User oppure null. Ma null non è nel tipo.

> Il tipo di ritorno è un contratto. Se il contratto mente,
> non hai un sistema di tipi — hai una decorazione.

ACT 2

Null in natura: 3 significati, 0 informazione

Non trovato

`getUser() → null`

Non caricato

`this.user == null`

Errore silenzioso

`process() → null`

Stesso tipo. Tre significati completamente diversi.

CREPA #3

Chi possiede cosa?

Ownership implicito e stato mutabile condiviso

ACT 2

Il problema

ServiceA → Config → ServiceB

ServiceB modifica Config ⚡

Nessuno ha dichiarato chi possiede quell'oggetto.
Chi ha il diritto di modificarlo.

ACT 2

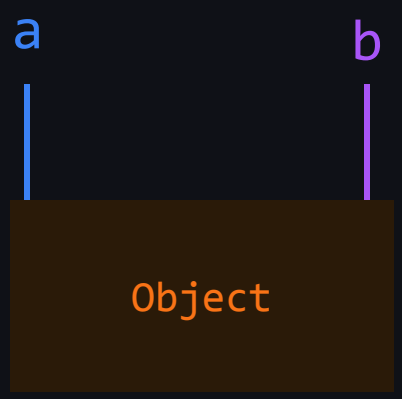
Il codice innocente

```
void processItems(List items) {
    items.sort()  // ordina in-place!
    // ... elabora ...
}
```

Il chiamante si aspettava che la lista rimanesse intatta. Non lo era.

ACT 2

Aliasing



Due nomi. Stesso oggetto.
Modifichi tramite a ⟶ la modifica si vede tramite b.

Java, Python, JavaScript, C# — comportamento di default.

ACT 2

Defensive copies

```
new ArrayList<>(original)
Collections.unmodifiableList(original)
// deep copy? shallow copy? clone()?
```

> Stai scrivendo codice per difenderti dal linguaggio,
> non per risolvere il problema del tuo utente.

ACT 2

Il vero costo

Ogni volta che leggi codice OOP con stato mutabile,
devi tenere in testa una mappa:

- ? Chi ha questo riferimento?
- ? Chi potrebbe averlo modificato?
- ? Quando?
- ? Da dove viene questo valore?

Overhead cognitivo costante. Non lo misuri con i profiler.

CREPA #4

Il contratto rotto

Le eccezioni

ACT 2

Checked exceptions: l'idea giusta

```
void readFile() throws IOException
```

L'idea era sana: rendere espliciti i fallimenti possibili.
Il compilatore controlla.

Era la cosa giusta. Ma il mercato l'ha rifiutata.

ACT 2

Perché il mercato le ha rifiutate

```
try {
    doSomething();
} catch (IOException e) {
    throw new RuntimeException(e);     // wrap & forget
}
```

Il boilerplate vince. Non per pigrizia — per necessità strutturale.

ACT 2

Il tipo di ritorno mente (di nuovo)

La firma

```
String processPayment(Order)
```

La realtà

→ NullPointerException
→ IllegalStateException
→ RuntimeException(IOException)
→ ...e altri 3 casi

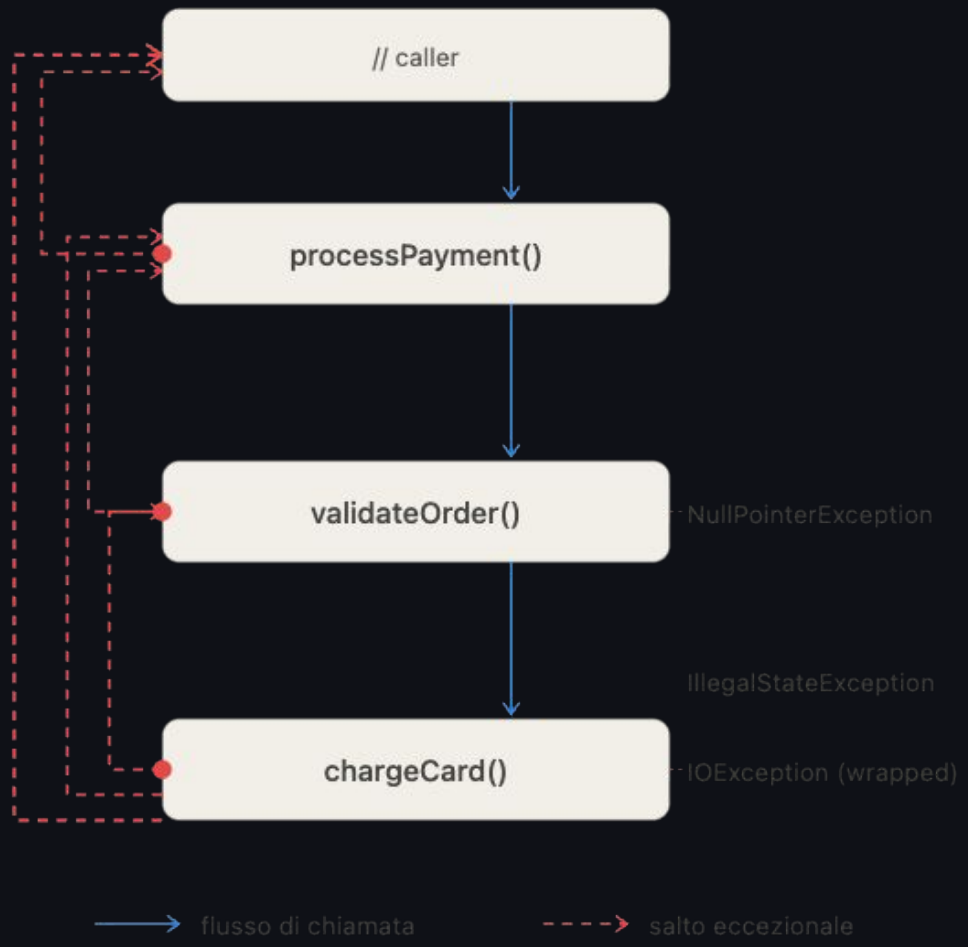
```
catch (Exception e) {
    // TODO
}
```

Tutti l'abbiamo scritto.

ACT 2

Il control flow invisibile

Il codice che leggi non ti dice come si comporta il sistema.
Ti dice come si comporta nel caso ottimale.



L'OOP ci ha insegnato a modellare il mondo.

Non ci ha insegnato a dire quando il modello è sbagliato.

Ci ha dato l'encapsulation.

Poi ha nascosto i bug dentro.

Ci ha dato l'inheritance.

Poi ci ha reso prigionieri delle scelte di chi è venuto prima.

Ci ha dato i pattern.

E ci ha convinto che la loro esistenza fosse eleganza —
non un segnale d'allarme.

Fred Brooks, 1986 — "No Silver Bullet"

La complessità accidentale

Essenziale

Viene dal problema stesso.
Non puoi eliminarla.

Accidentale

La aggiungiamo noi. Con le scelte
di design, astrazioni, paradigmi.

La cosa più pericolosa è quando smetti di riconoscerla come tale.

Non eravamo stupidi

L'OOP ha risolto problemi reali. Ha reso il codice più organizzato.
Ha permesso di costruire sistemi enormi con team enormi.

> Il problema non è l'OOP.
> Il problema è che abbiamo smesso di farci domande.

I pattern come sintomo

I pattern non sono la prova della potenza dell'OOP.
Sono la prova dei suoi limiti.

ACT 3

Strategy Pattern

Java classico

Interfaccia + implementazione
+ injection = ~30 righe

JS / Kotlin / Rust

```
fn(strategy: fn(x) → y)
```

Una riga. Funzione di prima classe.

Ogni Strategy in Java è una funzione di prima classe scritta a mano.

ACT 3

Builder Pattern

Java

```
User.builder()
  .name("Alice")
  .age(30)
  .build()
```

Kotlin / Swift / Python

```
User(name: "Alice", age: 30)
```

Named parameters + defaults.

Builder risolve l'assenza di named parameters nel linguaggio.

ACT 3

Adapter Pattern

Java — serve un wrapper

```
// libreria esterna: non modificabile
class ThirdPartyLogger {
    void writeLog(String msg) { ... }
}

// la nostra interfaccia
interface Logger {
    void log(String msg);
}

// adapter: esiste solo per colmare
// il gap tra due contratti
class LoggerAdapter implements Logger {
    private ThirdPartyLogger tpl;
    LoggerAdapter(ThirdPartyLogger t) {
        this.tpl = t;
    }
    public void log(String msg) {
        tpl.writeLog(msg); // delega
    }
}
```

Rust — impl diretto sul tipo esterno

```
// libreria esterna: non modificabile
struct ThirdPartyLogger;
impl ThirdPartyLogger {
    fn write_log(&self, msg: &str) { ... }
}

// il nostro trait
trait Logger {
    fn log(&self, msg: &str);
}

// impl del nostro trait
// direttamente sul tipo esterno
impl Logger for ThirdPartyLogger {
    fn log(&self, msg: &str) {
        self.write_log(msg);
    }
}
// nessun wrapper — zero boilerplate
```

In Rust — il problema che Adapter risolve non esiste.

ACT 3

Quanti pattern per "creare oggetti"?

- Factory
- Abstract Factory
- Prototype
- Builder
- Object Pool

23 pattern. Molti sono workaround.

La densità è un segnale.

Il punto

Un linguaggio maturo dovrebbe risolvere questi problemi
nel sistema dei tipi — non nella cultura.
Non in un libro di ricette.

> I pattern sono soluzioni a problemi reali.
> Ma alcuni di quei problemi sono stati creati dal paradigma stesso.

Cosa hanno fatto i linguaggi moderni

Le idee sono portabili.

ACT 4

Option<T>: null è un tipo

Java

```
User getUser(String id)
```

Può restituire null.
Il tipo non lo dice.

Rust

```
fn get_user() -> Option<User>
```

L'assenza è nel tipo.
Il compilatore controlla.

Non puoi ignorare il caso "non c'è". Non per disciplina — per costruzione.

ACT 4

Result<T, E>: gli errori sono valori

Java

Lancia eccezioni.
O restituisce null.
O parametro di output.

Rust

```
Result<Receipt, PaymentError>
```

Successo o fallimento.
Nel tipo. Obbligatorio.

> Gli errori sono valori. Non salti invisibili nel control flow.

Ownership: chi possiede cosa è nel tipo

```rust
let user = User::new();
process(user);        // ownership trasferito
println!("{}", user); // errore di compilazione!
```

Non ci sono alias impliciti. Non ci sono modifiche a sorpresa.
Non per disciplina — per garanzia del compilatore.

Traits vs Inheritance

```
impl Display for User { ... }
impl Serialize for User { ... }
impl Validate for User { ... }
```

20 trait. Nessuna gerarchia. Niente Fragile Base Class.
Polimorfismo senza ereditarietà.

Il consiglio del 1994, implementato a livello di linguaggio.

ACT 4

Non solo Rust

Kotlin

String vs String?, Result, sealed class

Swift

Optional built-in, pattern matching, Result

TypeScript

Union types, never, literal types

Java 17+

Optional<T>, record, sealed classes

Non dobbiamo imparare Rust. Dobbiamo applicare queste idee nel linguaggio che già usiamo.

ACT 4

Immutabilità come default

Rust

```
let x = 5;      // immutabile
let mut y = 5;  // opt-in
```

Java / JS / Python

```
x = 5;          // mutabile
final y = 5;    // opt-in
```

"Chi potrebbe aver modificato questo valore?" → Nessuno.

ACT 4

Composizione in pratica

Ereditarietà

Gerarchia, metodi ridefiniti, stato condiviso, comportamento accoppiato

Composizione + Traits

Tipi separati, comportamento esplicito, nessun accoppiamento

Più facile da testare. Più facile da modificare. Più facile da capire.

OOP non è sbagliato.

Ha un trentennio di successi reali.
Miliardi di righe in produzione che funzionano.

> C'è differenza tra "questo strumento ha dei limiti"
> e "questo strumento è inutile".
> Oggi ho parlato dei limiti.

ACT 5

Cosa disimparare

1. **extends come primo istinto**
   Serve davvero ereditarietà, o basta composizione?

2. **Null come valore legittimo**
   Usa Optional. Il tipo deve dire la verità.

3. **Eccezioni per il control flow**
   Gli errori previsti sono valori, non eccezioni.

4. **Stato mutabile condiviso**
   Chi possiede questo? Chi può modificarlo?

5. **I pattern come obiettivo**
   Se non hai il problema, non applicare la soluzione.

ACT 5

Tre domande da portarsi a casa

Stai modellando un "è un" o un "ha un"?

Se ambiguo → composizione.

La sottoclasse può sostituire la superclasse?

Liskov Substitution. Se no → la gerarchia è sbagliata.

Stai ereditando comportamento o riutilizzando codice?

Per riutilizzare codice, basta una funzione.

```
> rethinking inheritance...                    ✓ DONE

> making ownership explicit...                 ✓ DONE

> treating errors as values...                 ✓ DONE

> defaulting to immutability...                ✓ DONE

> unlearning...                          ▒▒▒▒▒▒ IN PROGRESS
```

L'ultimo non finisce mai. Ed è okay.

Risorse

- Design Patterns — Gamma, Helm, Johnson, Vlissides
- Effective Java — Joshua Bloch
- Programming Rust — Blandy, Orendorff, Tindall
- "Simple Made Easy" — Rich Hickey
- "OOP to Rust" — Enzo Lombardi

Grazie.

Domande? Obiezioni?
Difensori dell'ereditarietà multipla?
https://enzolombardi.net
https://www.linkedin.com/in/enzo-lombardi-ab8a19/

AITW

CONFERENCE

31 MARZO 2026 | ROMA

Don't forget to rate the talk!